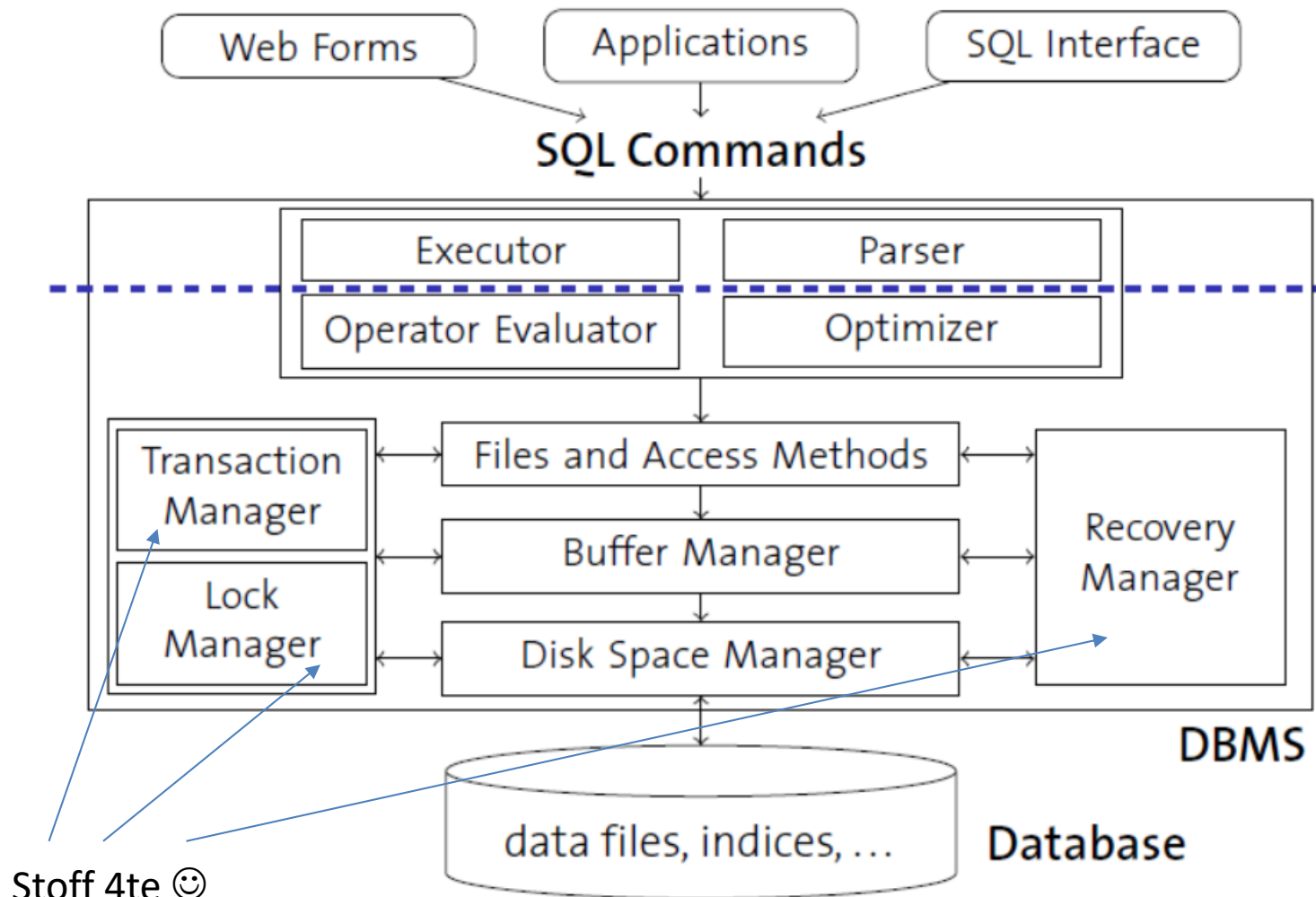


DBI 5

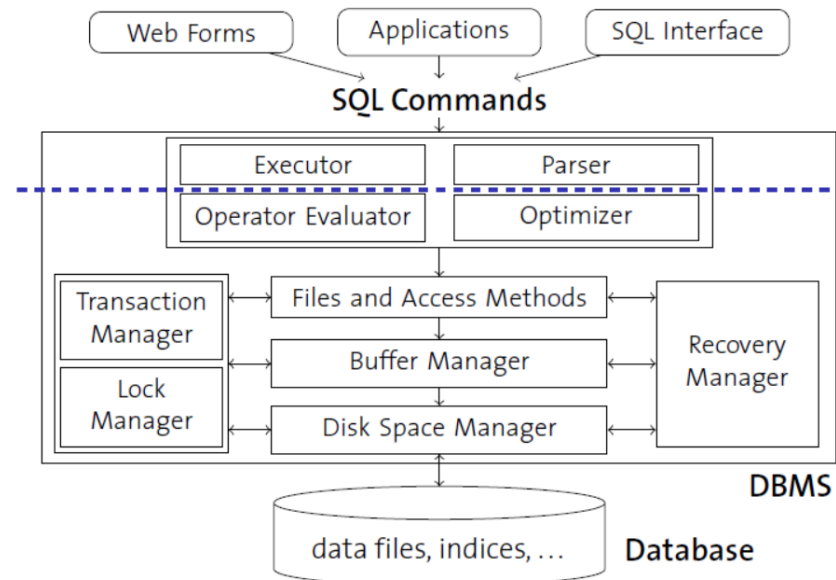
Architektur eines DBMS Oracle im Speziellen

Architektur eines DBMS



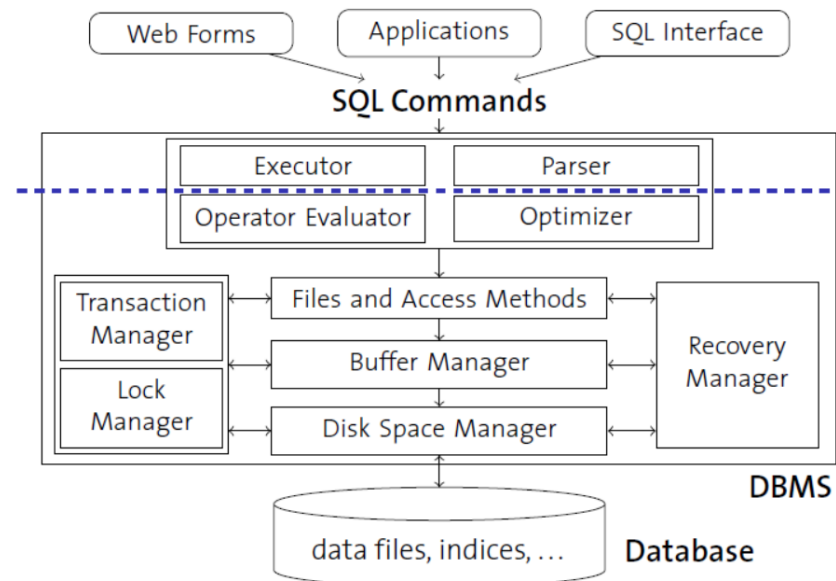
Disk Space Manager

- Verwaltet die Daten auf dem externen Speichermedium (z.B. Platte)
- Die höheren Schichten des DBMS sehen keine HW-Details, sondern nur noch eine Sammlung von Seiten (in der Größenordnung 4 -64 KB)
- Disk Space Manager stellt Routinen bereit für
 - Allokieren / Deallokieren von Seiten
 - Lesen / Schreiben einer Seite

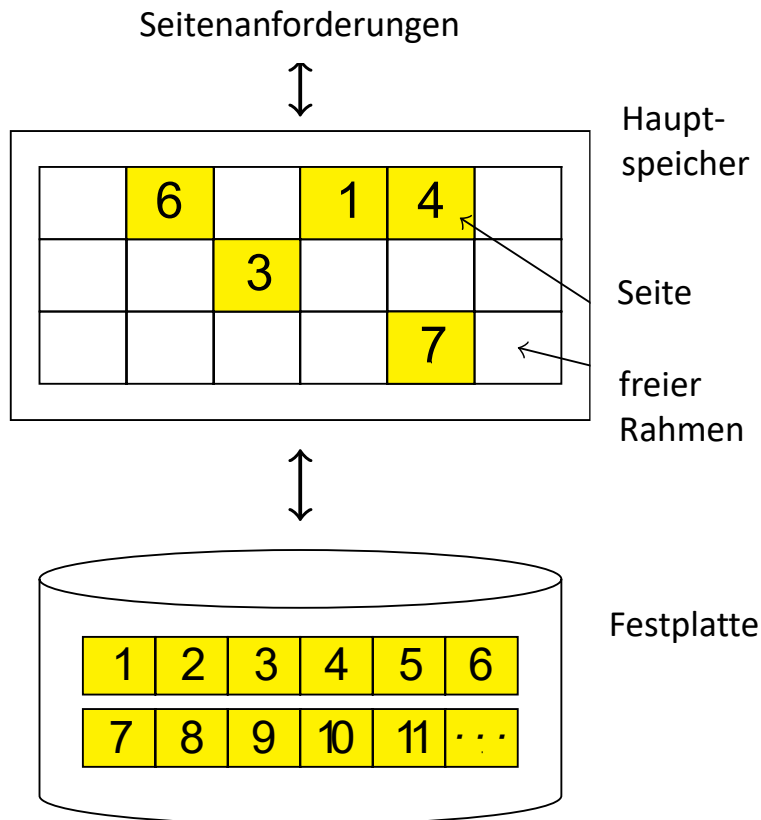


Buffer Manager

- Verwaltet den Datenbankpuffer (buffer pool) im Hauptspeicher
- Bearbeitung von Daten kann immer nur im Hauptspeicher (und nicht direkt auf der Platte) geschehen. ⇒ Seiten müssen von der Platte in den Datenbankpuffer gelesen und später wieder auf Platte zurückgeschrieben werden.
- Die anderen Schichten des DBMS fordern einfach eine Seite an.
- Buffer Manager speichert zu jeder Seite im Puffer Informationen:
 - ob die Seite noch verwendet wird: in diesem Fall darf die Seite nicht durch andere Seiten überschrieben werden.
 - ob die Seite geändert wurde (→ dirty page, die Daten der Page im Hauptspeicher entsprechen nicht mehr den Daten des externen Speichers): nur in diesem Fall ist das Zurückschreiben auf die Platte nötig
- Verdrängungsstrategie falls Puffer voll ist, z.B. Least Recently Used (LRU) Verdrängung der Seite mit der am längsten zurückliegenden Veränderung);



Puffer-Verwalter



- Vermittelt zwischen externem und internem Speicher (Hauptspeicher)
- Verwaltet hierzu einen besonderen Bereich im Hauptspeicher, den Pufferbereich (buffer pool)
- Externe Seiten in Rahmen des Pufferbereichs laden
- Verdrängungsstrategie falls Pufferbereich voll

Datenbanken vs. Betriebssysteme

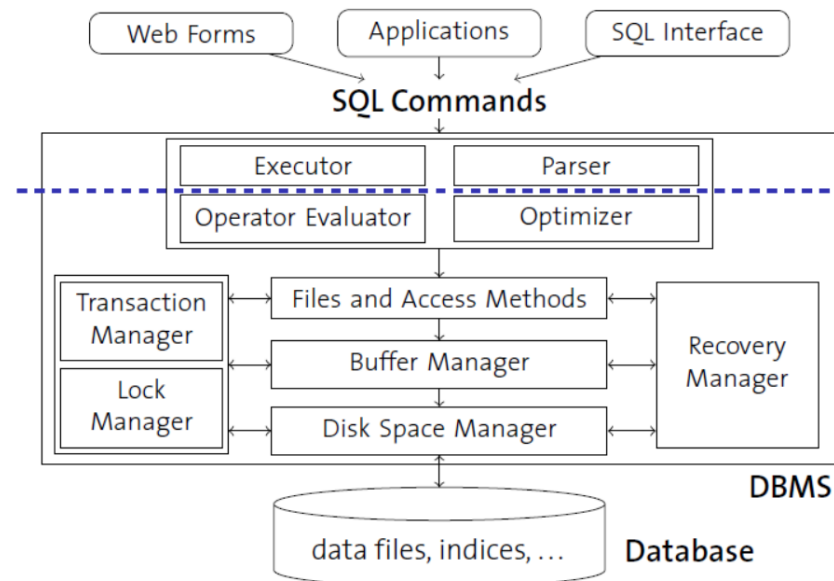
- Haben wir nicht gerade ein Betriebssystem entworfen?
- Ja
 - Verwaltung für externen Speicher und Pufferverwaltung ähnlich
 - Memory-mapped files
- Aber
 - DBMS weiß mehr über Zugriffsmuster (z.B. Prefetching)
 - Limitationen von Betriebssystemen häufig zu stark für DBMS (Obergrenzen für Dateigrößen, Plattformunabhängigkeit nicht gegeben)

Datenbanken vs. Betriebssysteme

- Gegenseitige Störung möglich
 - Doppelte Seitenverwaltung
 - DMBS-Transaktionen vs. Transaktionen auf Dateien organisiert vom Betriebssystem (journaling)
 - DBMS Pufferbereiche durch Betriebssystem ausgelagert auf Festplatte
- (Daher:) DBMSe schalten Betriebssystemdienste aus
 - Direkter Zugriff auf Festplatten
 - Eigene Prozessverwaltung
 - ...

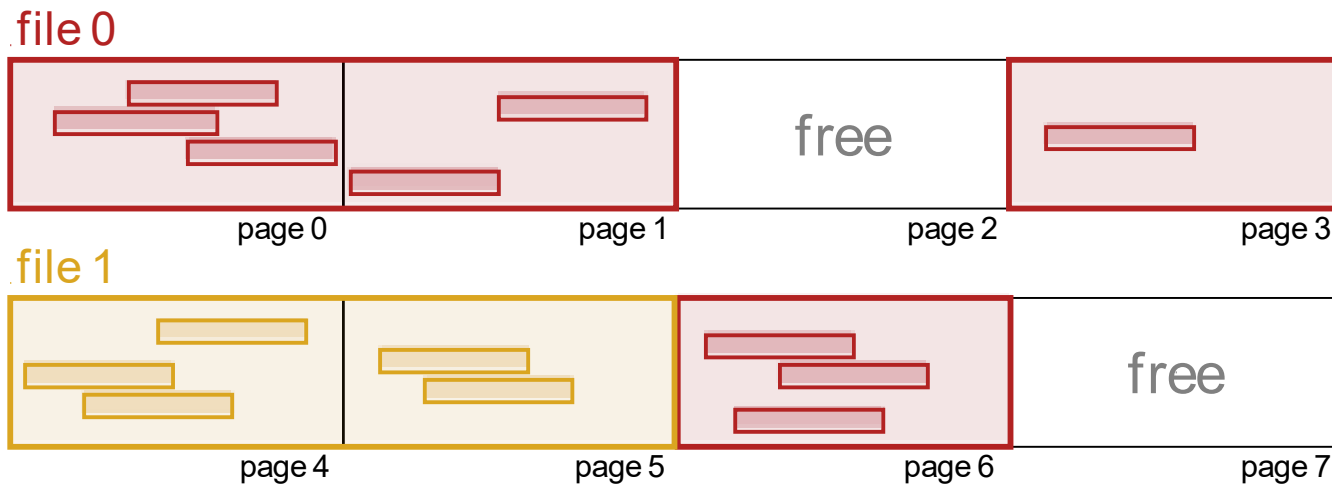
File and Access Methods

- Seitenverwaltung unbeeinflusst vom Inhalt
- Die Zeilen einer DB-Tabelle (= Tupel einer Relation) haben im Allgemeinen nicht auf einer einzelnen Seite Platz. Logisch zusammengehörige Seiten werden als "File" verwaltet.
- Zwei Hauptaufgaben dieses Software Layers:
 - Verwaltung der *Seiten eines Files* (entspricht üblicherweise einer Tabelle):
 - "Heap file": Zufällige Verteilung der Tupel auf die Seiten
 - "Index file": Spezielle Verteilung der Tupel auf die Seiten zwecks Opt. bestimmter Zugriffe.
 - Verwaltung der *Tupel* innerhalb einer einzelnen Seite



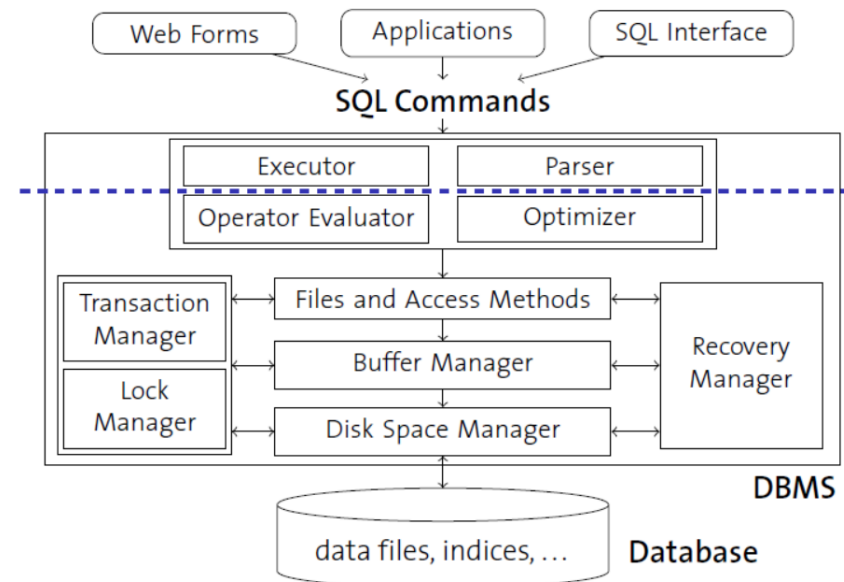
Datenbank-Dateien

- Seitenverwaltung ist unbeeinflusst vom Inhalt
- DBMS verwaltet aber auch Inhalte: Tabellen von Tupeln, Indexstrukturen, ...
- Tabellen sind Dateien von Datensätzen (records)
 - Datei besteht aus einer oder mehrerer Seiten
 - Jede Seite speichert eine oder mehrere Datensätze
 - Jeder Datensatz korrespondiert zu einem Tupel



Operator Evaluator

- bei der Evaluierung von Abfragen wird geprüft, wie hoch der Aufwand ist, um eine Abfrage beantworten zu können.
- Ein DBMS sollte mit minimalen Ressourcen große Datenmengen so schnell wie möglich bearbeiten können



Anfragebeantwortung

Projektion

Aggregation

```
SELECT C.CUST_ID, C.NAME, SUM (O.TOTAL) AS REVENUE
FROM CUSTOMERS AS C, ORDERS AS O
WHERE C.ZIPCODE BETWEEN 8000 AND 8999
      AND C.CUST_ID = O.CUST_ID
GROUP BY C.CUST_ID
ORDER BY C.CUST_ID, C.NAME
```

Restriktion

Join

Gruppierung

Sortierung

Ein DBMS muss eine Menge von Aufgaben erledigen:

- mit minimalen Ressourcen
- über großen Datenmengen
- und auch noch so schnell wie möglich

Optimizer

legt die geeignetste Strategie für den Zugriff auf die Daten und legt somit den Ausführungsplan fest.

- a) *regelbasiertem Optimizer*: folgen einem Regelwerk, e.g. wenn vorhanden Index verwenden, ohne Informationen über die in den Tabellen gespeicherten Daten, e.g. die Anzahl der Datensätze in den beteiligten Tabellen, Werte, die besonders häufig vorkommen, der Sortierzustand der Sätze.
- b) *kostenbasiertem Optimizer*: Erstellen alle möglichen Ausführungsplan-Varianten, wenden ein Kostenmodell auf jede Variante an und wählen schlussendlich die Variante mit den besten Kosten.
 - Vergleichen zum Beispiel die Kosten eines Ausführungsplanes mit Index zu denen eines Ausführungsplanes ohne Index (voller Tabellenzugriff).

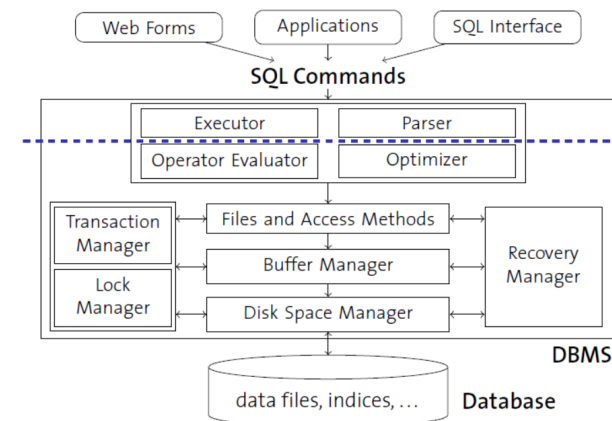
Die meisten Optimizer sind kostenbasierte - damit ist hauptsächlich die Rechenzeit gemeint.

Hauptaufgaben des Optimizers sind:

- die Auswahl des Join Algorithmus und der Join Reihenfolge
- die Verwendung von Indizes

Optimizer sind nicht zuständig für:

- Optimierung von Tabellen oder Indizes
- Optimierung von verstümmeltem SQL
- Daten Defragmentierung



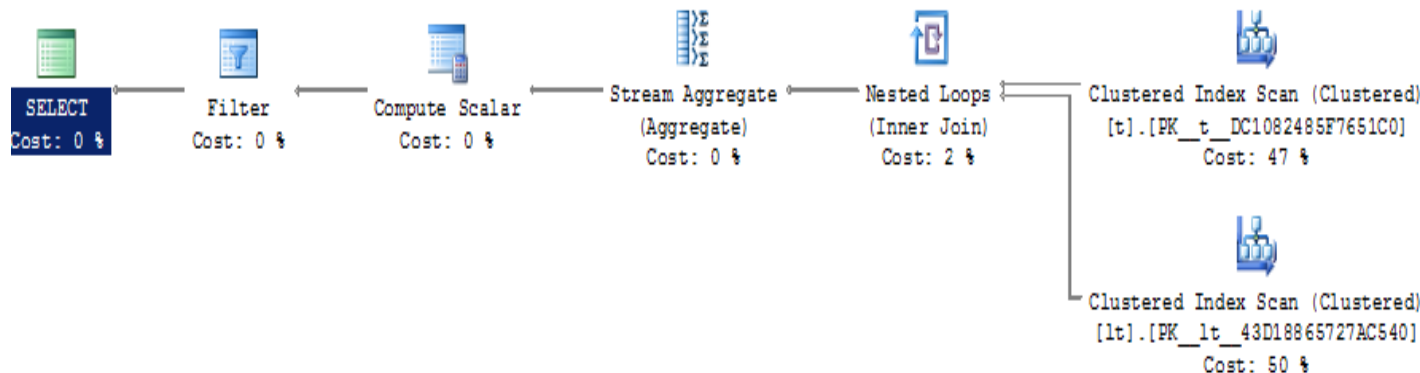
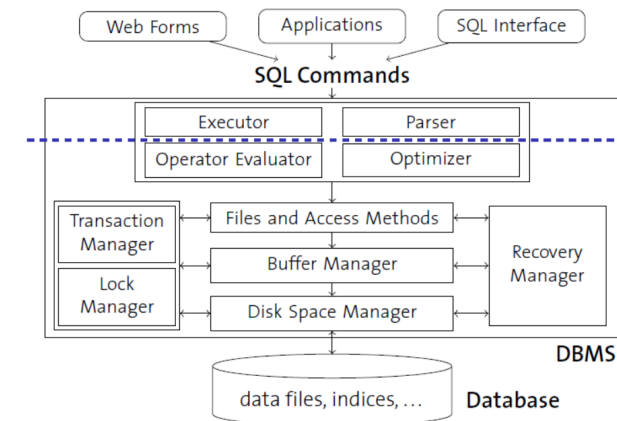
Executor (Ausführungsplan)

Dabei handelt es sich um eine Beschreibung, in welchen Einzelschritten ein relationales Datenbankmanagementsystem eine Datenbankabfrage ausführt und in welcher Reihenfolge dies geschieht. Der Ausführungsplan wird vom Optimizer erstellt.

Beispiel:

```
select t.tnr, t.tname, sum(lt.menge) as gesliefermenge
from lt join t on lt.tnr = t.tnr
where t.farbe = 'rot'
group by t.tnr, t.tname having count(lt.lnr) > 1;
```

SQL Server:



Executor (Ausführungsplan)

Beispiel:

```
select t.tnr,t.tname,sum(lt.menge)as gesliefermenge
from lt join t on lt.tnr = t.tnr
where t.farbe = 'rot'
group by t.tnr,t.tname having count(lt.lnr) > 1;
```

Oracle Server:

PLAN_TABLE_OUTPUT

Plan hash value: 2053954590

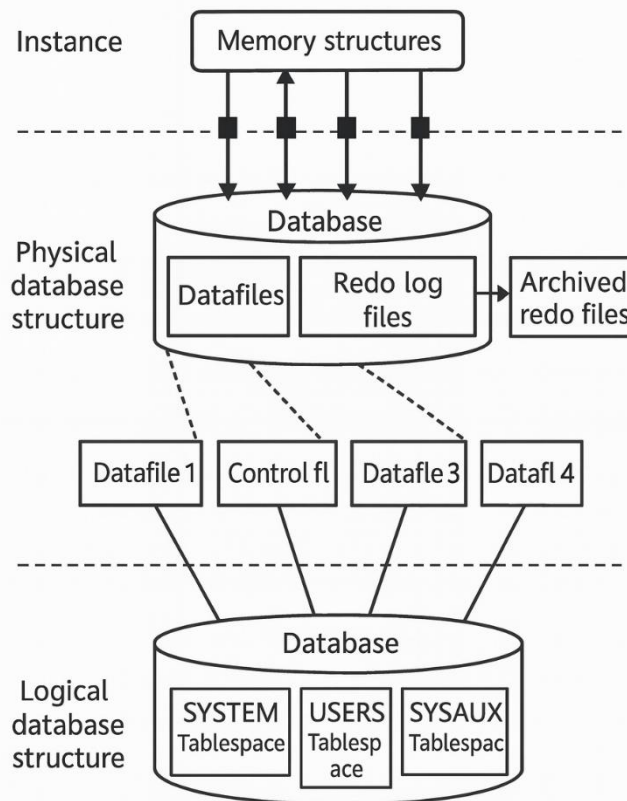
Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		6	228	8 (25)	00:00:01
* 1	FILTER					
2	HASH GROUP BY		6	228	8 (25)	00:00:01
* 3	HASH JOIN		6	228	7 (15)	00:00:01
* 4	TABLE ACCESS FULL	T	3	63	3 (0)	00:00:01
5	TABLE ACCESS FULL	LT	12	204	3 (0)	00:00:01

Predicate Information (identified by operation id):

-
- 1 - filter(COUNT(*)>1)
 - 3 - access("LT"."TNR"="T"."TNR")
 - 4 - filter("T"."FARBE"='rot')

Oracle

Oracle Database 23c



- Oracle Instanz (*Instance*):
- Systemprozesse, z.B. *PMON*, *DBW0*
- Benutzerprozesse (Datenbankverbindungen)
- Speicherstrukturen (im RAM), z.B. *SGA*
- Pro Instanz *eine* Oracle Datenbank (*Database*):
- Physische Datenbankstruktur (Datenfiles, Controlfile, Redo-Log-Dateien)
- Logische Datenbankstruktur (Tablespaces, Segmente, Extents, Blocks)

Logische Datenbankstruktur

Tablespace: Bereich, in dem Daten wie Tabellen oder Indizes gespeichert werden. Besteht aus einem oder mehreren *Datenfiles*

- Standardinstallation enthält fünf Tablespaces: SYSTEM, SYSAUX, UNDOTBS1, TEMP, USERS
- Eine Oracle Datenbank besteht aus mindestens zwei Tablespaces:
 - SYSTEM: Enthält den Systemkatalog.
 - SYSAUX: Enthält Daten, die dem Systembereich zuzuordnen sind, aber nicht zum Kernbereich der Datenbank gehören.

Segment: physische Bereich in einem Tablespace, der durch eine Tabelle (Datensegment) oder einen Index (Indexsegment) belegt wird.

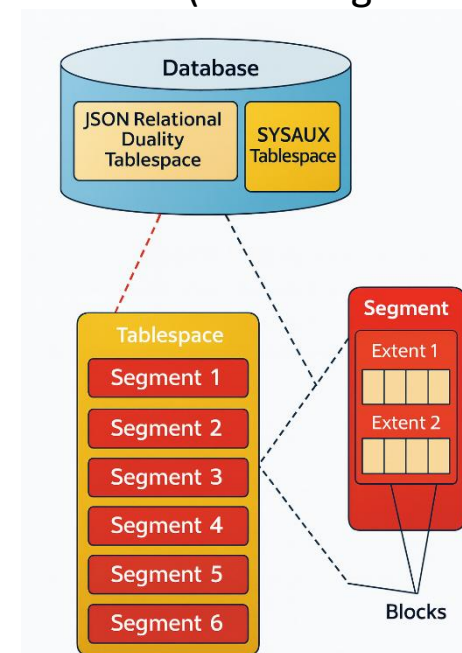
Es gibt auch noch temporäre Seg. und Undo-Seg.

Extent: zusammengehörige Gruppe von Datenbankblöcken, die physisch ohne Zwischenraum gespeichert werden. Ein Extent ist immer vollständig in einem bestimmten Datenfile gespeichert.

Block: kleinste Einheit bei I/O Ops der DB.

Meist ein Vielfaches der OS-Blockgröße.

Gängige für Oracle sind 8192 Bytes



Physische Datenbankstruktur

Datenfile: entspricht einer physischen Datei im OS und ist Teil von *genau einem* Tablespace. Ein Tablespace kann aus mehreren. Kann automatisch wachsen via AUTOEXTEND-Klausel. Häufig benutzte Blöcke von Datenfiles werden im RAM gehalten (Cache). Neue und geänderte Datenblöcke werden verzögert auf die Disk zurückgeschrieben.

Redo Log File: Protokollieren Datenänderungen. Muss mindestens zwei geben, die als Ringbuffer verwendet werden. Wenn ein Server den Speicherinhalt (RAM) verliert, kann aus den Informationen darin wieder ein konsistenter Zustand der Datenbank hergestellt werden. Diese Dateien sind mit den *Transaktions-Logdateien* im *Microsoft SQL Server* vergleichbar.

Control Files: enthält Metadaten über den Aufbau der Datenbank, e.g. die Namen und Pfade aller Datenfiles und Redo Log Dateien. Geht es verloren, sind die Daten in der Datenbank nicht mehr zugänglich!

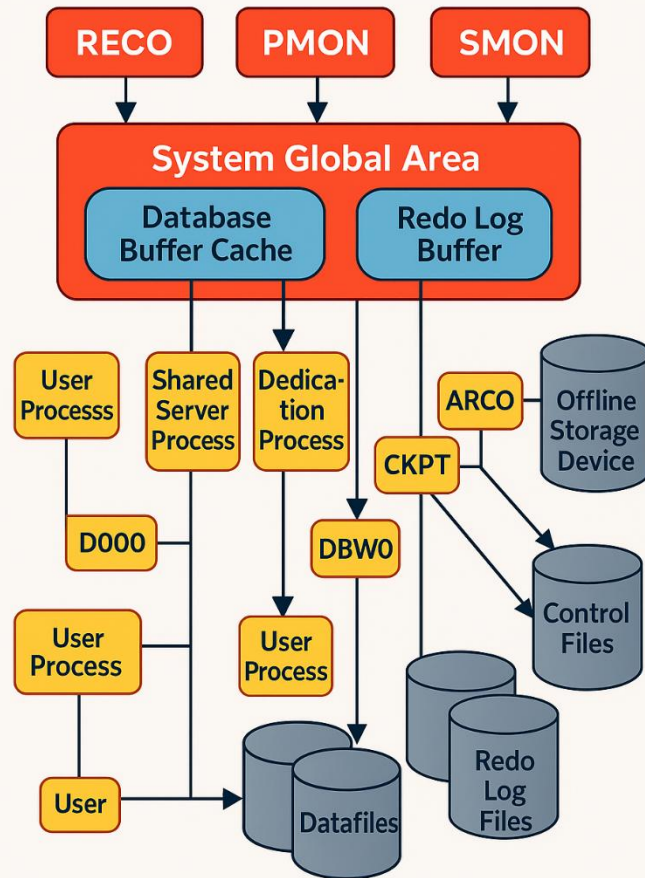
Archive Log Files:

- Im NOARCHIVELOG werden die Redo Log Dateien nach und nach überschrieben=> Bei einem Totalverlust der Datenfiles kann der Datenstand nur bis zum letzten Backup wiederhergestellt werden.
- Im ARCHIVELOG werden die Redo Log Dateien an spezielle Speicherorte kopiert und von dort häufig gesichert. Ist nach einem Totalverlust noch zumindest ein Exemplar der archivierten Logdateien vorhanden, kann mit ihrem Inhalt, den Redo Log Dateien und dem letzten Backup der Datenstand bis exakt zum Zeitpunkt des Totalverlustes wiederhergestellt werden.

Dateien mit Initparametern für wichtige Einstellungen: spifle/pfile

Alert und Trace Log Dateien: Für Fehler im Betrieb (BACKGROUND_DUMP_DEST) oder Verhaltensaufzeichnungen von Usern (USER_DUMP_DEST)

Systemprozesse



SMON: Crash Recovery nach Systemabsturz,
Zusammenlegen von freiem Speicherplatz

PMON: Räumt nicht mehr nötige DB-Verbindungen und
zugehörige Ressourcen auf

DBWn: Schreiben neue oder geänderte Blöcke (*Dirty Blocks*)
vom *Buffer Cache* in die Datenfiles, Checkpoints

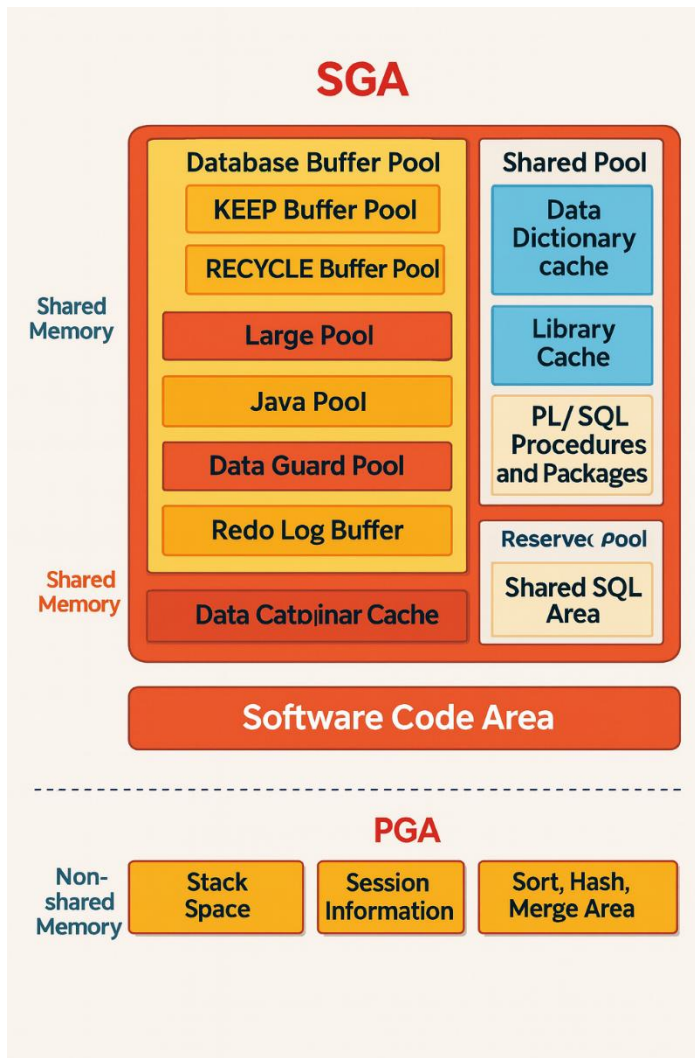
LGWR: Schreibt Änderungsinformationen in die Redo Log
Files

ARCn: Kopieren Redo Log Dateien als Archived Log Dateien
an einen sicheren Ort (für Wiederherstellung)

CKPT: Aktualisiert die Header von Control- und Datenfiles
mit der SCN (*System Change Number*)

RECO: Behandelt Fehler bei *verteilten Transaktionen*.

Speicherstrukturen



System Global Area (SGA)

Database buffer cache: puffert kürzlich gelesene Datenbankblocks im RAM (Performance)

Shared Pool: Besteht aus *library cache* (Execution plane für SQL und PL/SQL) und *data dictionary cache* (Puffer für Systemkatalog).

Redo Log Buffer: Enthält die letzten Datenblockänderungen. Wird vom LGWR in die Datenfiles geschrieben (all 3s, >1MB, >1/3 voll).

Large Pool Optional: für verteilte Transaktionen, parallele Abfragen, RMAN paralleles Backup und Restore.

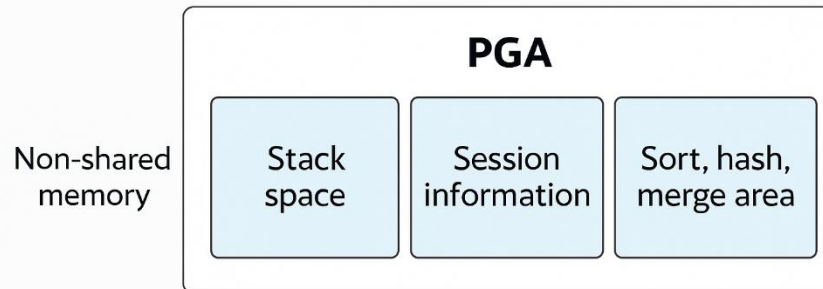
Java Pool Optional: Verwendet für die Oracle Java Virtual Machine. In Oracle Datenbanken kann Java code ablaufen.

Streams Pool Optional: Speicher für verteilte Umgebungen (Oracle Streams).

Program Global Area (PGA)

Software Code Area

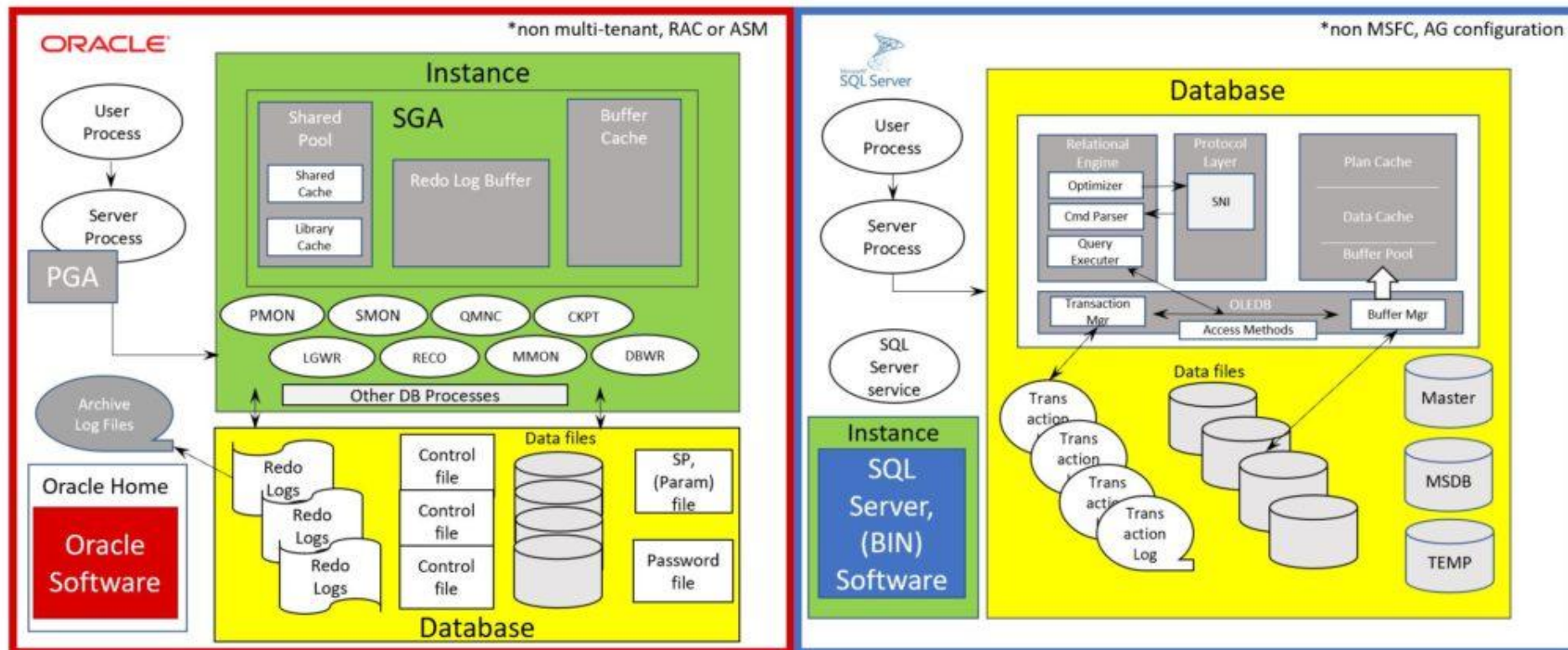
- Enthält die Speicherbereiche für Benutzerverbindungen im *dedicated server*-Betrieb (Sessioninformation). Es gibt auch einen Bereich für Sortieroperationen (*sort area*).



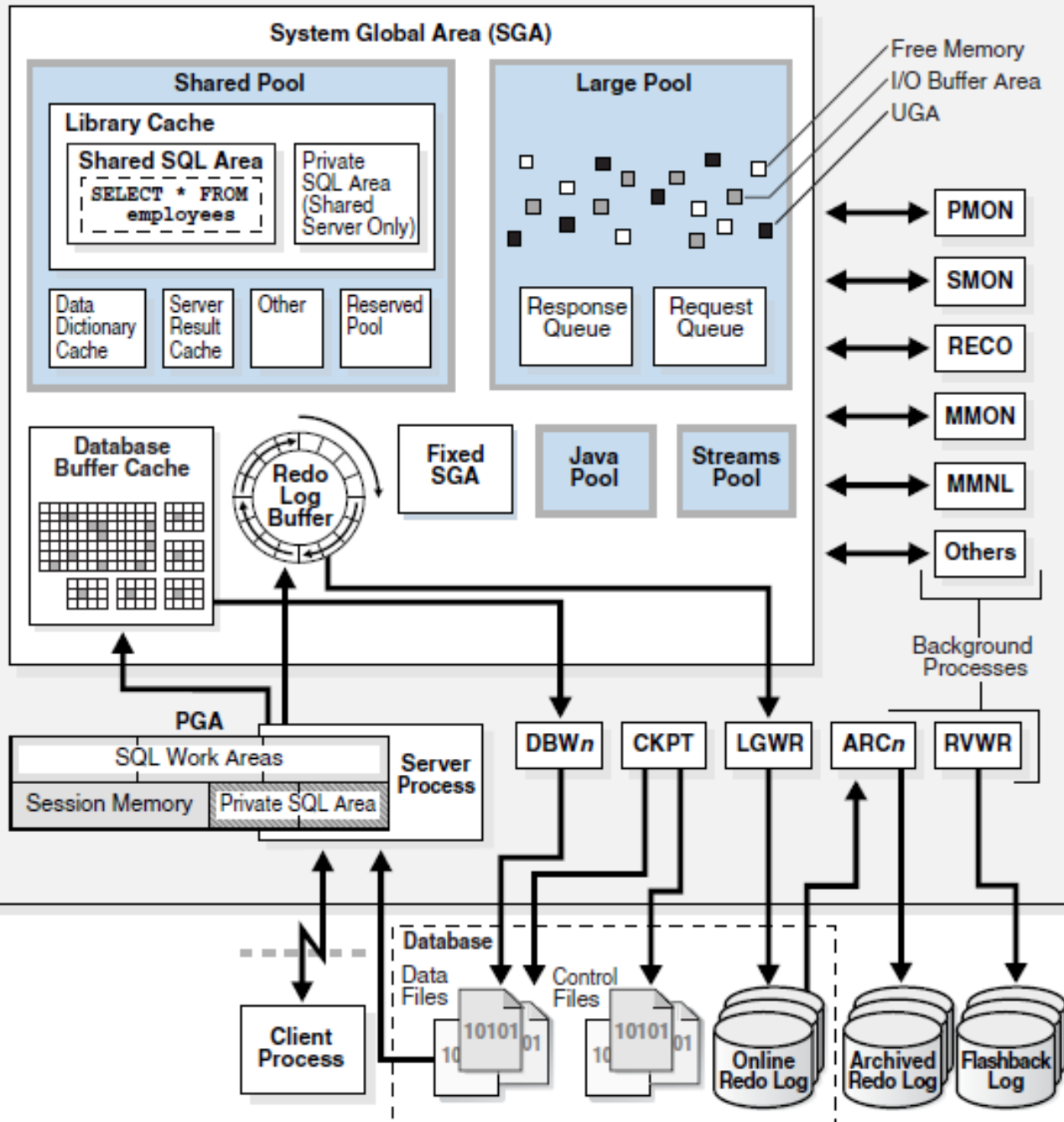
- Software Code Area: Enthält den ausführbaren Programmcode der Oracle-Software.

Oracle vs Microsoft SQL Server

Oracle Compared to SQL Server, (High Level)



Update – was passiert?



- Client Prozess: übermittelt UPDATE inkl. Commit
- Server Prozess: Parsen, lesen der betroffenen DB Blöcke in Buffer Cache, ändern der Blöcke
- LGWR: schreiben der Änderungsinformationen vom Redo Log Buffer ins Online Redo Log. Erst dann Bestätigung des Commit
- DBWn: Schreibt die geänderten Blöcke vom Buffer Cache in die Datenfiles
- ARCn: Kopiert vollgeschriebene Online Redo Logs an einen sicheren Ort

Systembenutzerkonten

Zur Administration der Oracle Servers gibt es zwei Benutzerkonten:

- **SYSTEM:** hat die Rolle DBA und das SYSDBA-Systemrecht. Alle Systemtabellen und Objekte gehören diesem Benutzerkonto.
- **SYS:** hat zusätzlich das Recht zum Start und Stoppen der Datenbank.

Instanzen starten/stoppen

als SYS eingeloggt sein, um die Instanz zu stoppen oder zu starten.

Stoppen der Instanz:

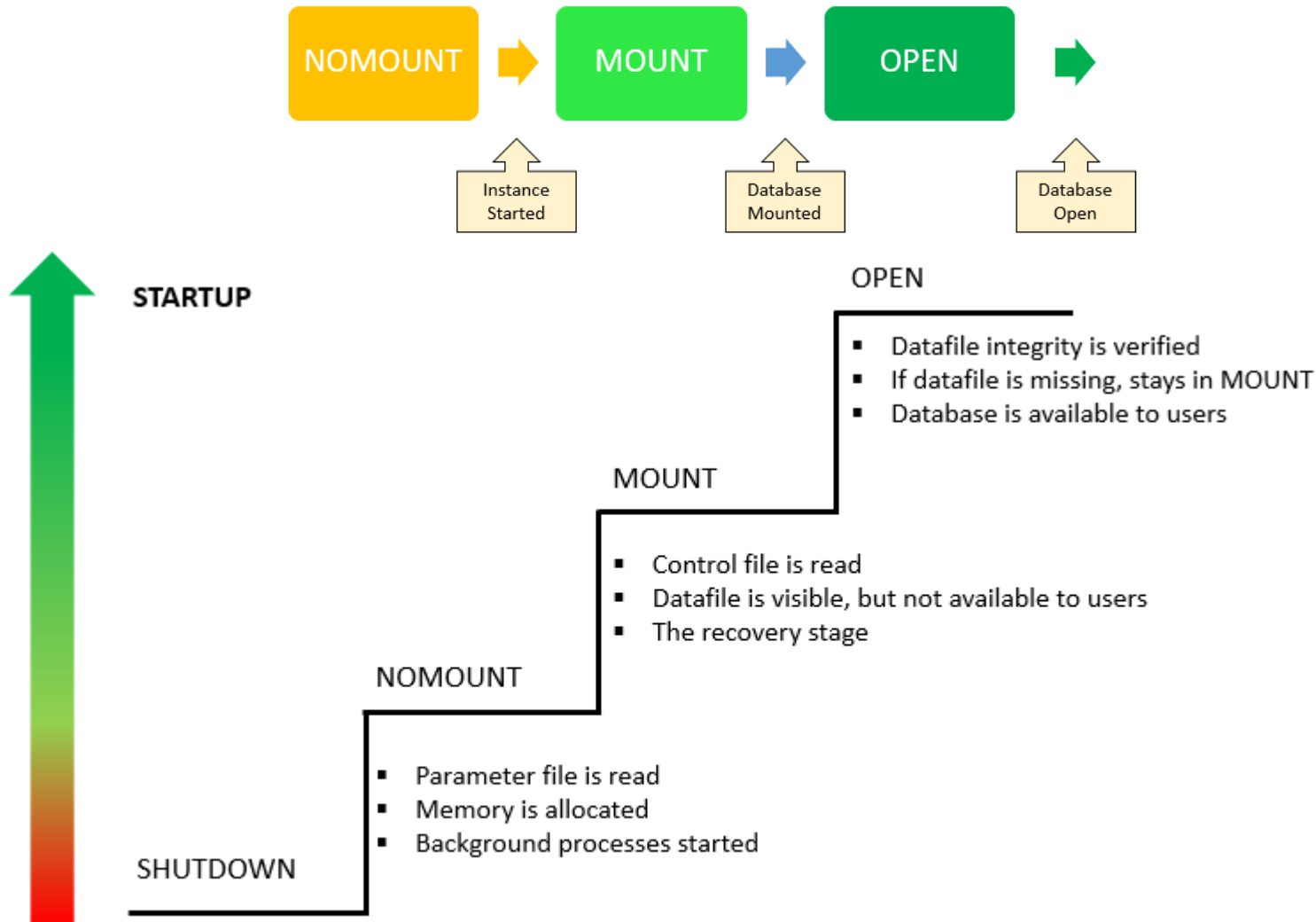
- shutdown, shutdown normal: Wartet, bis sich alle Benutzer abgemeldet haben. Neue Datenbankverbindungen sind nicht erlaubt.
- shutdown immediate: Alle aktiven Transaktionen werden abgebrochen und zurückgerollt, die Clientverbindungen werden beendet, neue Verbindungen sind nicht mehr erlaubt.
- shutdown abort: Alle Transaktionen werden abgebrochen (*kein* Rollback), die Clientverbindungen werden beendet. Recovery und Rollback werden beim nächsten Start durchgeführt. **Nur in Notfällen verwenden.**

Starten der Instanz:

- startup: Instanz wird gestartet und Datenbank geöffnet (Normalfall).
- startup restrict: Nur Benutzer mit Recht RESTRICTED SESSION können sich anmelden. Späteres alter system disable restricted session nötig.
- startup nomount: Instanz wird gestartet, Controlfile jedoch nicht gelesen. Anschließendes alter database mount nötig.
- startup mount: Instanz wird gestartet und Controlfile gelesen. Datenfiles sind nun bekannt, aber nicht geöffnet. Anschließendes alter database open nötig.

startup nomount und startup mount werden hauptsächlich für Wartungsarbeiten benötigt.

Startup phase



Systemkatalog

- herstellerspezifischen **Systemkatalog (Data Dictionary)**
- *keine* ISVs (Information Schema Views) wie in der SQL Norm vorgesehen
- Ein Namenspräfix gibt an, welche Datenbankobjekte in einer Tabelle sichtbar sind (am Beispiel von <Prefix>_TABLES):
 - USER_TABLES: Alle Tabellen, die der Benutzer besitzt.
 - ALL_TABLES: Alle Tabellen, auf die der Benutzer Rechte hat.
 - DBA_TABLES: Alle Tabellen einer Datenbank.
 - CDB_TABLES: Tabellen aller Datenbanken in der Container-Datenbank.
- Welche Tabellen abgefragt werden können, hängt von der Benutzerrolle ab.

